

Statistics for a data rich world—some explorations*

Stefano Allesina (adapted by Lin Chen and Xuanyao Liu) *University of Chicago*

Goal: More and more often we need to analyze large and complex data sets. However, the statistical methods we've been taught in college have evolved in a data-poor world. Modern biology requires new tools, which can cope with the new questions and methods that arise in a data-rich world. Here we are going to discuss problems that often arise in the analysis of large data sets. We're going to review hypothesis testing (and what happens when we have many hypotheses) and discuss model selection. We're going to see the effects of selective reporting and p-hacking, and how they contribute to the *reproducibility crisis* in the sciences. **Audience:** Biologists with some programming background.

I. Review of hypothesis testing

Statistics is the science of collecting and analyzing data from samples in order to estimate and make inference regarding the the population. A census data is often not feasible, and also not necessary. A sample is a smaller and random subset of the target population and the sample is collected to represent the population. Hypothesis testing is a major component of statistical inference.

The basic idea of hypothesis testing is the following: we have devised an hypothesis on our system, which we call H_1 (the “alternative hypothesis”). We have collected our data, and we would like to test whether the data are consistent (or inconsistent) with the so-called “null hypothesis” (H_0). The null hypothesis is a contradiction to the alternative hypothesis.

The simplest example is that of a bent coin: my friend Aiden likes to bet on a coin toss with people, and he often chooses head and wins. I suspect that he has a bent coin in favor of head (H_1 : the coin is bent). We therefore toss the coin several times and check whether the number of heads we observe is consistent with the null hypothesis of a fair coin (H_0 : the coin is a fair coin).

In R we can toss many coins in no time at all. Call p the probability of obtaining a head, and initially toss a fair coin ($p = 0.5$) a thousand times:

First, when simulating a data, we always want to set seed to make sure the results are reproducible.

```
set.seed(222)
p <- 0.5 # probability of a head (fair coin)
flips <- 1000 # number of times we flip the coin
data <- sample(c("H", "T"),
              flips, prob = c(p, 1 - p),
              replace = TRUE)
heads <- sum(data == "H")
```

*This document is included as part of the ‘Statistics for a data rich world’ tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2026. **Current version:** August 13, 2026.

There is also a faster way to flip the coins and count the heads by assuming that the number of heads out of 1000 flips follows a binomial distribution:

```
heads <- rbinom(1, flips, p)
```

If the coin is fair, we expect approximately 500 heads, but of course we might have small variations due to the randomness of the process. We therefore need a way to distinguish between “bad luck” and an incorrect hypothesis.

What is customarily done is to compute the probability of recovering the observed or a more extreme version of the pattern under the null hypothesis: if the probability is very small, it implies that when the null hypothesis is true, there is a very small probability (i.e., very unlikely) that you can observe things as or more extreme than what you have observed in the current data. We call this probability a *p-value*. A small *p-value* (typically less than 0.05) indicates strong evidence against the null hypothesis, so you reject the null hypothesis and conclude that the data is inconsistent with the null hypothesis. A large *p-value* (> 0.05) only means that when the null hypothesis is true, you are likely to observe what has been observed in the current data but that is not enough to prove the null hypothesis is true, so you fail to reject the null hypothesis and the conclusion is inconclusive. When the null hypothesis is indeed true or when the alternative hypothesis is true but you do not have enough power to reject the null, you may end up with a large *p-value*.

For example, if the coin is fair, the number of heads should follow the binomial distribution. The probability of observing a larger number of heads than what we’ve got is therefore

```
one.sided.pvalue <- 1 - pbinom(heads, flips, 0.5)
```

Here and in the following sections, we calculated a one-sided *p-value* testing $H_0 : p = 0.5$ versus $H_A : p > 0.5$. Note that in many other cases, we recommend a two-sided test, and a two-sided *p-value* together with a 95% confidence interval can be obtained via:

```
heads <- rbinom(1, flips, p)
pvalue <- binom.test(heads, flips, 0.5, alternative="two.sided")
```

What if we repeat the tossing many, many times?

```
# flip 1000 coins 1000 times, and count the number of heads in each of the 1000 experiments
# produce histogram of number of heads
heads_distribution <- rbinom(1000, flips, p)
hist(heads_distribution, main = "distribution number heads", xlab = "number of heads")
```

You can see that it is very unlikely to get more than 560 (or less than 440) heads when flipping a fair coin 1000 times. Therefore, if we were to observe say 400 heads (or 600), we would tend to believe that the coin is biased (though of course this could have happened by chance if we are repeating the tossing 1 trillion times!).

Type I and type II errors

When testing an hypothesis, we can make two types of errors:

- **Type I error:** reject H_0 when it is in fact true. Also known as false positive.
- **Type II error:** fail to reject H_0 when in fact it is not true. Also known as false negative.

We call α the probability of making a type I error (or type I error rate), and β as type II error rate. And power is in fact $1 - \beta$. We can calculate the p-value based on the data and compare the p-value with the significance threshold α . The p-value quantifies how strongly the data contradicts the null hypothesis, and if $p < \alpha$, we reject the null hypothesis. Type I and Type II error rates are inversely related. In choosing a significance level for a test, you are actually deciding how much you want to risk committing a type I error — rejecting the null hypothesis when it is. The more stringent α is (0.01 versus 0.05) in controlling type I error rate, the less likely you would make a rejection decision and consequently the power will be reduced too.

The distribution of p-values

Suppose that we are tossing each of several fair coins 1000 times. For each, we compute the corresponding (one-sided) p-value testing the null hypothesis $p = 0.5$ against the alternative $p > 0.5$. How are the p-values distributed?

```
ncoins <- 2500
heads <- rbinom(ncoins, flips, p)
pvalues <- 1-pbinom(heads, flips, 0.5)
hist(pvalues, xlab = "p-value", freq = FALSE)
abline(h = 1, col = "red", lty = 2)
```

As you can see, if the data were generated under the null hypothesis, the distribution of the p-values would be approximately uniform between 0 and 1. This means that if we set $\alpha = 0.05$, we would reject the null hypothesis 5% of the time (even though in this case we know the hypothesis is correct!).

What is the distribution of the p-values if we are tossing biased coins? We will find an enrichment in small p-values, with stronger effects for larger biases:

```
p <- 0.52 # the coin is biased
heads <- rbinom(ncoins, flips, p)
pvalues <- 1 - pbinom(heads, flips, 0.5)
hist(pvalues, xlab = "p-value", main = paste0("p = ", p), freq = FALSE)
abline(h = 1, col = "red", lty = 2)

p <- 0.55 # the coin is biased
heads <- rbinom(ncoins, flips, p)
pvalues <- 1 - pbinom(heads, flips, 0.5)
hist(pvalues, xlab = "p-value", main = paste0("p = ", p), freq = FALSE)
abline(h = 1, col = "red", lty = 2)
```



C. elegans

II. The challenges with p-values

Selective reporting

Articles reporting positive results are easier to publish than those containing negative results. Authors might have little incentive to publish negative results, which could go directly into the file-drawer.

This tendency is evidenced in the distribution of p-values in the literature: in many disciplines, one finds a sharp decrease in the number of tests with p-values just below 0.05 (which is customarily—and arbitrarily—chosen as a threshold for “significant results”). For example, we find many a sharp decrease in the number of reported p-values of 0.051 compared to 0.049—while we expect the p-value distribution to decrease smoothly.

Selective reporting leads to irreproducible results: we always have a (small) probability of finding a “positive” result by chance alone. For example, suppose we toss a fair coin many times, until we find a “significant” result.

On the other hand, more and more journals would require us to report effect size estimates and confidence intervals – “Were this procedure to be repeated on numerous samples, the fraction of calculated confidence intervals (which would differ for each sample) that encompass the true population parameter would tend toward 90%.”(source: Cox D.R., Hinkley D.V. (1974) Theoretical Statistics, Chapman & Hall, p49, p209.)

Problem: p-hacking

The problem is well-described by Simonsohn et al. (J. Experimental Psychology, 2014): “While collecting and analyzing data, researchers have many decisions to make, including whether to collect more data, which method to use, which measure(s) to analyze, which covariates to use, what to do with outliers and missing data, and so on. If these decisions are not made in advance but rather are made as the data are being analyzed, then researchers may make them in ways that self-servingly increase their odds of publishing. Thus, rather than placing entire studies in the file-drawer, researchers may file merely the subsets of analyses that produce nonsignificant results. We refer to such behavior as *p-hacking*.” The term p-hacking describes the conscious or subconscious manipulation of data in a way that produces a desired p-value.

The same authors showed that with careful p-hacking, almost anything can become significant (read their hilarious article in [Psychological Science](#), where they show that listening to a song can change the listeners’ age!).

Discussion on p-values

Selective reporting and p-hacking are only two of the problems associated with the widespread use and misuse of p-values. The discussion in the scientific community on this issue is extremely topical. I have collected some of the articles on this problem in the readings folder. Importantly, in 2016 the American Statistical Association released a statement on p-values every scientist should read.

Reproducibility crisis

P-values and hypothesis testing contribute considerably to the so-called *reproducibility crisis* in the sciences. A survey promoted by *Nature* magazine found that “More than 70% of researchers have tried and failed to reproduce another scientist’s experiments, and more than half have failed to reproduce their own experiments.”

This problem is due to a number of factors, and addressing it will likely be one of the main goals of science in the next decade.

Exercise: p-hacking

Go to goo.gl/a3UOEF and try your hand at p-hacking, showing that your favorite party is good (bad) for the economy.



D. rerio

III. Multiple comparisons (also known as multiple testing)

The problem of multiple comparisons arises when we perform multiple statistical tests. Since each test is subject to some small chance of producing a false positive result, when jointly considering many many tests, the chances of producing some false positive findings are much higher.

Suppose we perform our coin tossing exercise, flipping 1000 coins 1000 times each. For each coin, we determine whether our data differs significantly from what expected by contrasting our p-value with a significance level $\alpha = 0.05$.

Even if the coins are all perfectly fair, we would expect to find approximately $0.05 \cdot 1000 = 50$ coins that lead to the rejection of the null hypothesis.

In fact, we can calculate the probability of making at least one type I error (reject the null when in fact it is true). This probability is called the Family-Wise Error Rate (FWER). It can be computed as 1 minus the probability of making no type I error at all. If we set $\alpha = 0.05$, and assume the tests to be independent, the probability of making no errors in m tests is $1 - (1 - 0.05)^m$. Therefore, if we perform 10 tests, we have about 40% probability of making at least a mistake; if we perform 100 tests, the probability grows to more than 99%. If the tests are not independent, we can still say that in general $FWER \leq m\alpha$.

This means that setting an α per test does not control for FWER.

Moving from tossing coins to biology, consider the following examples:

- **Gene expression** In a typical RNAseq experiment, we compare the differential expression levels of tens of thousands of genes in the treatment and control tissues.
- **GWAS** In Genome-Wide Association Studies we want to find single-nucleotide polymorphisms (SNPs) associated with a given phenotype. It is common to test millions of SNPs for significant associations.
- **Identifying binding sites** Identifying candidate binding sites for a transcriptional regulator requires scanning the whole genome, yielding tens of millions of tests.

Organizing the tests in a table

Suppose that we're testing m hypotheses. Of these, an unknown subset m_0 is true, while the remaining $m_1 = m - m_0$ are false. We would like to correctly call the true/false hypotheses (as much as possible). We can summarize the results of our tests in a table, of which the elements are unobservable:

What we would like to know is $m_1 = T + S$ and $m_0 = U + V$. Then V is the number of type I errors (rejected H when in fact it is true), and T is the number of type II errors (failed to reject a false H). However, we can only observe $V + S$ (the number of "discoveries"), and $U + T$ (number of "failures").

The type I error rate is $E[V]/m$ (where $E[X]$ stands for expectation). When there are many tests (m) being considered, controlling for type I error rates at 0.05 means that by random chance, one could make $0.05 \times m$ false positive findings even if all m tests are under the null. For example, when testing genetic associations between 10 million genetic variants to the risk of a disease, if still using 0.05 as the significance threshold, there could be 500k significant findings by random chance even if the disease is not heritable and has no genetic association. Apparently, we need to choose a much more stringent significance threshold to account for the number of tests, and there are some other error measures. The Family-wise error rate is defined as $P(V > 0)$. Another quantity of interest is the False Discovery Rate (FDR), measured as the proportion of true discoveries $FDR = E[V/(V + S)]$ when $V + S > 0$. FDR measures the proportion of falsely rejected hypotheses.

Importantly, FWER guards against any single false positive finding among all m tests, and is a more stringent significance criteria than FDR in multiple comparison problems. It also means that when we control for FWER, we're automatically controlling for the FDR, but not vice versa. It should be noted that when a large number of tests is performed, controlling FWER could be quite conservative and may lose power.

Methods for multiple testing correction: Bonferroni correction

One of the simplest and most widely-used procedures to control for FWER is Bonferroni's correction. This procedure controls for FWER in the case of independent or dependent tests. It is typically quite conservative, especially when the tests are not independent (in practice, it becomes "too conservative" when the number of tests is moderate to high). Fundamentally, for a desired FWER α we choose as a significance threshold of α/m for each single test, where m is the number of tests we're performing. Equivalently, we can "adjust" the p-values as $q_i = \min(m \cdot p_i, 1)$, and call significant the values $q_i < \alpha$. In R it is easy to perform this correction:

```
original_pvals <- c(0.012, 0.06, 0.77, 0.001, 0.32)
adjusted_pvals <- p.adjust(original_pvals, method = "bonferroni")
print(adjusted_pvals)
```

With these adjusted p-values, and an $\alpha = 0.05$, we would still single out as significant the fourth test, but not the first. The strength of Bonferroni is its simplicity, and the fact that we can perform the operation in a single step. Moreover, the order of the tests does not matter.

Other procedures: the Holm–Bonferroni method (or the Holm method)

There are several refinements of Bonferroni's correction, some of which use the sequence of ordered p-values. For example, the Holm method starts by sorting the p-values in increasing order $p_{(1)} \leq p_{(2)} \leq p_{(3)} \leq \dots p_{(m)}$. The hypothesis $H_{(i)}$ is rejected if $p_{(j)} \leq \alpha / (m - j + 1)$ for all $j = 1, \dots, i$. Equivalently, we can adjust the p-values as $q_{(i)} = \min(1, \max((m - i + 1)p_{(i)}, q_{(i-1)}))$. In this way, we use the most stringent threshold to determine whether the smallest p-value is significant, the next smallest p-value uses a slightly higher threshold and so on. The Holm method is uniformly more powerful than the Bonferroni correction.

For example, using the same p-values above:

```
original_pvals <- c(0.012, 0.06, 0.77, 0.001, 0.32)
adjusted_pvals <- p.adjust(original_pvals, method = "holm")
print(adjusted_pvals)
```

We see that we would be calling the first test significant, contrary to what obtained with Bonferroni.

The function `p.adjust` offers several choices for p-value correction. Also, the package `multcomp` provides a quite comprehensive set of functions for multiple hypothesis testing.

An example: testing mixed coins

We're going to test these concepts by tossing repeatedly many coins. In particular, we're going to toss 1000 times 50 biased coins ($p = 0.55$) and 950 fair coins ($p = 0.5$). For each coin, we're going to compute a p-value, and count the number of type I, type II, etc. errors when using unadjusted p-values as well as when correcting using the Bonferroni or Holm procedure.

```
toss_coins <- function(p, flips){
  # toss a coin with probability p of landing on head several times
  # return a data frame with p, number of heads, pval and
  # H0 = TRUE if p = 0.5 and FALSE otherwise
  heads <- rbinom(1, flips, p)
  pvalue <- 1 - pbinom(heads, flips, 0.5)
  if (p == 0.5){
    return(data.frame(p = p, heads = heads, pval = pvalue, H0 = TRUE))
  } else {
    return(data.frame(p = p, heads = heads, pval = pvalue, H0 = FALSE))
  }
}

# To ensure everybody gets the same results, we're setting the seed
set.seed(8)
data <- data.frame()
# the biased coins
for (i in 1:50) data <- rbind(data, toss_coins(0.55, 1000))
# the fair coins
```

```
for (i in 1:950) data <- rbind(data, toss_coins(0.5, 1000))
# here's the data structure
head(data)
```

Now we write a function that adjusts the p-values and builds the table above

```
get_table <- function(data, adjust, alpha = 0.05){
  # produce a table counting U, V, T and S
  # after adjusting p-values for multiple comparisons
  data$pval.adj <- p.adjust(data$pval, method = adjust)
  data$reject <- FALSE
  data$reject[data$pval.adj < alpha] <- TRUE
  return(table(data[,c("reject", "H0")]))
}
```

First, let's see what happens if we don't adjust the p-values:

```
no_adjustment <- get_table(data, adjust = "none", 0.05)
print(no_adjustment)
```

We correctly declared 48 of the biased coins “significant”, but we also incorrectly called 2 biased coins “not significant” (Type II error). More worryingly, we called 45 fair coins biased when they were not (Type I error). To control for the family-wise error rate, we can correct using Bonferroni:

```
bonferroni <- get_table(data, adjust = "bonferroni", 0.05)
print(bonferroni)
```

With this correction, we dramatically reduced the number of type I errors (from 45 to 0), but at the cost of increasing type II errors (from 2 to 40) and losing power. In this way, we would make only 10 discoveries instead of 50.

In this case, Holm's procedure does not help:

```
holm <- get_table(data, adjust = "holm", 0.05)
print(holm)
```

More sophisticated methods, for example the Benjamini-Hochberg (BH) procedure based on controlling false discovery rate ($FDR = E(\text{false discoveries} / \text{significant tests})$, where E stands for expectation), can reduce the type II errors and improve power, at the cost of a few and estimable type I errors:

```
BH <- get_table(data, adjust = "BH", 0.05)
print(BH)
```



C. jacchus

FDR and q-values

Inspired by the need for controlling for FDR in genomics, Storey and Tibshirani (PNAS 2003) have proposed the idea of a q-value, measuring the probability that a feature that we deemed significant turns out to be not significant after all.

One uses p-values to control for the false positive rate (# false positive / total test): when determining significant p-values we control for the rate at which null features in the data are called significant. The False Discovery Rate (# false positive / # significant test), on the other hand, measures the rate at which results that are deemed significant are truly null. While setting $PCER = 0.05$ we are stating that about 5% of the truly null features will be called significant, an $FDR = 0.05$ means that among the features that are called significant, about 5% will turn out to be null.

They proposed a method that uses the ensemble of p-values to determine the approximate (local) FDR. The idea is simple. If you plot your histogram of p-values when you have few true effect, and many nulls, you will see something like:

```
hist(data$pval, breaks = 25)
```

where the right side of the histogram is close to a uniform distribution. We could use the high p-values to find how tall the histogram would be if all effects were null, thereby estimating the proportion of truly null features $\pi_0 = m_0/m$.

Storey has built an R-package for this type of analysis:

```
# To install:
#install.packages("devtools")
#library("devtools")
#install_github("jdstorey/qvalue")
library("qvalue")
qobj <- qvalue(p = data$pval)
```

Here's the estimation of the π_0

```
hist(qobj)
```

which is quite good (in this case we know that $\pi_0 = 0.95$). The small p-values under the dashed line represent our false discoveries. Even better, through randomizations one can associate a q-value to each test, representing the probability of making a mistake when calling a result significant (formally, the q-value is the minimum FDR that can be attained when calling that test significant).

For example:

```
table((qobj$pvalues < 0.05) & (qobj$qvalues < 0.05), data$H0)
```

Note that the estimation of FDR is unstable if the denominator (# significant test) is expected to be small. Therefore, you may notice that FDR was widely used in detecting differentially expressed genes in diseased versus normal samples where the expected number of non-null tests is large. In contrast, in GWAS, researchers use the Bonferroni-adjusted p-value threshold of 5×10^{-8} to declare significance.

IV. Linear regression, logistic regression, and model selection

Linear regression

In statistics, linear regression is an approach to model the linear relationship between one response variable (or dependent variable) and one or more explanatory variables (or independent variables, or predictor). If there is only one explanatory variable, it is called simple linear regression. If the linear regression involves more than one explanatory variable, it is a multiple linear regression.

```
# create fake data for a simple linear regression
set.seed(5)
x <- 1:20
y <- 3 + 0.5 * x + rnorm(20)
plot(y ~ x)
```

We can fit a simple linear regression to the data

```
model1 <- lm(y ~ x)
summary(model1)
plot(y~x)
points(model1$fitted.values~x, type = "l", col = "blue")
```

In a data rich world, often, we need to select a model out of a set of reasonable alternatives with different combinations and/or (even nonlinear) patterns of explanatory variables. However, we run the risk of overfitting the data (i.e., fitting the noise as well as the pattern). The best fitted model for the current data from the current sample may not be the best model representing the pattern in the population of interest. Here is a simple example of a overfitted regression:

For the above data, we can also fit a more complex polynomial function of x .

```
model2 <- lm(y ~ poly(x, 7))
```

Let's see the residuals etc.

```
summary(model1)
summary(model2)
plot(y~x)
points(model1$fitted.values~x, type = "l", col = "blue")
points(model2$fitted.values~x, type = "l", col = "red")
```

Our second model has a much greater R^2 , i.e., more variation in the response variable can be explained by the model, but the second model also has many more parameters. The first model is more parsimonious. Which is a model we should choose?

Model selection tries to address this and similar problems. Most model fitting and model selection procedures are based on likelihoods (e.g., Bayesian models, maximum likelihood, minimum description length). The likelihood $L(\theta|D, M)$ is (proportional to) the probability of observing the data D under the model M and parameters θ . Because likelihood can be very small when you have much data, typically one works with log-likelihoods. For example:

```
logLik(model1)
logLik(model2)
```

Typically, more complex models will yield better (less negative) log-likelihoods and a better fit for the current data. However, more parameters would also increase the variation of the model. A complex model may not best represent the population (not to say computational burden). We will need to find a balance between bias and variance. We therefore want to penalize more complex models in some way.

AIC

One of the simplest methods to select among competing models is the Akaike Information Criterion (AIC). It penalizes models according to the **number of parameters**: $AIC = 2p - 2 \log L(\theta|D, M)$, where p is the number of parameters. Note that **smaller** values of AIC stand for “better” models. In R you can compute AIC using:

```
AIC(model1)
AIC(model2)
```

As you can see, AIC would favor the first (and simpler) model, which is also the model we used to simulate the data.

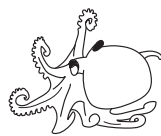
AIC is rooted in information theory and measures (asymptotically) the loss of information when using the model instead of the data. There are several limitations of AIC: a) it only holds asymptotically (i.e., for very large data sets; for smaller data you need to correct it); it penalizes each parameter equally (i.e., parameters that have a large influence on the likelihood have the same weight as parameters that do not influence the likelihood much); it can lead to overfitting, favoring more complex models in simulated data generated by simpler models.

BIC

In a similar vein, BIC (Bayesian Information Criterion) uses a slightly different penalization: $BIC = \log(n)p - 2 \log L(\theta|D, M)$, where n is the number of data points. You may see that for large data, BIC penalizes a complex model more than AIC. Again, smaller values stand for “better” models:

```
BIC(model1)
BIC(model2)
```

Here in this simple example, AIC and BIC agree with each other, and the simpler model is favored.



O. bimaculoides

Logistic regression

Logistic regression is often used to model the nonlinear relationship (modeled by a logistic function) between a binary response variable and a linear combination of explanatory variables. When the outcome is binary, for example pass/fail, win/lose, alive/dead, yes/no, one would consider a logistic regression. It can be extended to model several classes of a categorical variable as response.

We will start with an example. Fox et al. (Research Integrity and Peer Review, 2017) analyzed the invitations to review for several scientific journals, and found that “The proportion of invitations that lead to a submitted review has been decreasing steadily over 13 years (2003–2015) for four of the six journals examined, with a cumulative effect that has been quite substantial”. Their data is stored in `../data/FoxEtAl.csv`. We’re going to build models trying to predict whether a reviewer will agree (or not) to review a manuscript.

```
# read the data
reviews <- read.csv("../data/FoxEtAl.csv", sep = "\t")
# take a peek
head(reviews)
# set NAs to 0
reviews[is.na(reviews)] <- 0
# how big is the data?
dim(reviews)
# that's a lot! Let's take 5000 review invitations for our explorations;
# we will fit the whole data set later
set.seed(101)
small <- reviews[order(runif(nrow(reviews))),][1:5000,]
```

The response variable of interest is a reviewer i agreeing to review a manuscript or not, and is binary; and so we decided to use a logistic regression. Call π_i the probability that a reviewer i agree to review a manuscript. We model $\text{logit}(\pi_i) = \log(\pi_i / (1 - \pi_i))$ as a linear function.

Constant rate

As a null model we build a model in which the probability of agreeing to review does not change in time/for journals:

```
# suppose the rate at which reviewers agree is a constant
mean(small$ReviewerAgreed)
# fit a logistic regression
model_null <- glm(ReviewerAgreed~1, data = small, family = "binomial")
summary(model_null)
# interpretation:
exp(model_null$coefficients[1]) / (1 + exp(model_null$coefficients[1]))
```

Declining trend

We now build a model in which the probability to review declines steadily from year to year:

```
# Take 2003 as baseline
model_year <- glm(ReviewerAgreed~I(Year - 2003), data = small, family = "binomial")
#The I() function acts to convert the argument to "as.is"
summary(model_year)
```

Journal dependence

Reviewers might be more likely to agree for more prestigious journals:

```
# Take the first journal as baseline
model_journal <- glm(ReviewerAgreed~Journal, data = small, family = "binomial")
summary(model_journal)
```

Model journal and year

Finally, we can build a model combining both features: we fit a parameter for each journal/year combination

```
# Take the first journal as baseline, the colon mark stands for interaction term only.
model_journal_yr <- glm(ReviewerAgreed~Journal:I(Year-2003),
                        data = small, family = "binomial")
#summary(model_journal_yr)
```



T. thermophila

Likelihoods

In R, you can extract the log-likelihood from a model object calling the function `logLik`

```
logLik(model_null)
logLik(model_year)
logLik(model_journal)
logLik(model_journal_yr)
```

Interpretation: because we're dealing with binary data, the likelihood is the probability of correctly predicting the agree/not agree for all the 5000 invitations considered. Therefore, the probability of guessing a (random) invitation correctly under the first model is:

```
exp(as.numeric(logLik(model_null))) / 5000)
```

while the most complex model yields

```
exp(as.numeric(logLik(model_journal_yr))) / 5000)
```

We didn't improve our guessing much by considering many parameters! This could be due to specific data points that are hard to predict, or mean that our explanatory variables are not sufficient to model our response variable.

AIC

We can also calculate AIC for logistic models. In R you can compute AIC using:

```
AIC(model_null)
AIC(model_year)
AIC(model_journal)
AIC(model_journal_yr)
```

As you can see, the model `model_journal_yr` has the smallest AIC among all and is preferred here.

BIC

In a similar vein, BIC (Bayesian Information Criterion) uses a slightly different penalization: $BIC = \log(n)p - 2\log L(\theta|D, M)$, where n is the number of data points. Again, smaller values stand for "better" models:

```
BIC(model_null)
BIC(model_year)
BIC(model_journal)
BIC(model_journal_yr)
```

Note that according to BIC, `model_year` is favored. As mentioned before, BIC would penalize a complex model more.

Cross validation

One very robust method to perform model selection, often used in machine learning, is cross-validation. The idea is simple: split the data in three parts: a small data set for exploring; a large set for fitting; a small set for testing (for example, 5%, 75%, 20%). You can use the first data set to explore freely and get inspired for a good model. The data will be then discarded. You use the largest data set for accurately fitting your model(s). Finally, you validate your model or select over competing models using the last data set.

Because you haven't used the test data for fitting, this should dramatically reduce the risk of overfitting. The downside of this is that we're wasting precious data. There are less expensive methods for cross validation, but if you have much data, or data are cheap, then cross-validation has the virtue of being fairly robust.

Let's try our hand at cross-validation. First, we split the data into three parts:

```
reviews$cv <- sample(1:3, nrow(reviews), prob = c(0.05, 0.75, 0.2), replace = TRUE)
dataexplore <- reviews[reviews$cv == 1,]
datafit <- reviews[reviews$cv == 2,]
datatest <- reviews[reviews$cv == 3,]
# We've already done our exploration.
# Let's fit the data using model_journal
# and model_journal_yr, which seem to be the most promising
cv_model1 <- glm(ReviewerAgreed~I(Year-2003), data = datafit, family = "binomial")
cv_model2 <- glm(ReviewerAgreed~Journal:I(Year-2003), data = datafit,
  family = "binomial")
```

Now that we've fitted the models, we can use the function predict to find the fitted values for the testdata:

```
mymodel <- cv_model1
# compute probabilities
pi <- predict(mymodel, newdata = datatest, type = "resp")
# compute log likelihood
mylogLik <- sum(datatest$ReviewerAgreed * log(pi) +
  (1 - datatest$ReviewerAgreed) * log(1 - pi))
print(mylogLik)
```

repeat for the other model

```
mymodel <- cv_model2
# compute probabilities
pi <- predict(mymodel, newdata = datatest, type = "resp")
# compute log likelihood
mylogLik <- sum(datatest$ReviewerAgreed * log(pi) +
  (1 - datatest$ReviewerAgreed) * log(1 - pi))
print(mylogLik)
```

Cross validation supports the choice of the more complex model here.



S. aegyptiacus

Other approaches

Bayesian models are gaining much traction in biology. The advantage of these models is that you can get a posterior distribution for the parameter values, reducing the need for p-values and AIC. The downside is that fitting these models is computationally much more expensive (you have to find a distribution of values instead of a single value).

There are three main ways to perform model selection in Bayesian models:

- **Reversible-jump MCMC** You build a Monte Carlo Markov Chain that is allowed to “jump” between models. You can choose a prior for the probability of being in each of the models; the posterior distribution gives you an estimate of how much the data supports each model. Upside: direct measure. Downside: difficult to implement in practice – you need to avoid being “trapped” in a model.
- **Bayes Factors** Ratio between the probability of two competing models. Can be computed analytically for simple models. Can also be interpreted as the average likelihood when parameters are chosen according to their prior (or posterior) distribution. Upside: straightforward interpretation — it follows from Bayes theorem; Downside: in most cases, one needs to approximate it; can be tricky to compute for complex models.
- **DIC** Similar to AIC and BIC, but using distributions instead of point estimates.

Another alternative paradigm for model selection is Minimum-Description Length. The spirit is that a model is a way to “compress” the data. Then you want to choose the model whose total description length (compressed data + description of the model) is minimized.

A word of caution

The “best” model you’ve selected could still be a terrible model (best among bad ones). Out-of-fit prediction (such as in the cross-validation above) can give you a sense of how well you’re modeling the data.

When in doubt, remember the (in)famous paper in *Nature* by Tatem et al. 2004, which used some flavor of model selection to claim that, according to their linear regression, in the 2156 Olympics the fastest woman would run faster than the fastest man. One of the many memorable letters that ensued reads:

Sir — A. J. Tatem and colleagues calculate that women may out-sprint men by the middle of the twenty-second century (*Nature* 431,525; 2004). They omit to mention, however, that (according to their analysis) a far more interesting race should occur in about 2636, when times of less than zero seconds will be recorded.

In the intervening 600 years, the authors may wish to address the obvious challenges raised for both time-keeping and the teaching of basic statistics.

Kenneth Rice

Prediction for population outside the current data is prophecy.

V. Programming Challenge

P-hacking COVID-19

To show firsthand how p-hacking and overfitting are possible, we want you to show how these practices can lead to completely nonsensical results.

You can download a complete list of data on COVID-19 (coronavirus) by Our World in Data (<https://ourworldindata.org/coronavirus>). The data is updated daily and contains the latest publicly available data on COVID-19 by country and by date. The data report the total cases, new cases, total deaths, new deaths, and hospitalization data of 233 countries and regions. Note that you are not expected to analyze the entire data. You may choose one or a few countries, or select one or some dates for analysis or for comparison.

The challenge is to build an analysis pipeline that produces a “significant” p-value for a relationship between COVID-19 cases and another variable, where the relationship is non-sensical, cannot possibly be causal, or could be argued either way. You may even simulate a fake variable as your key variable of interest. Prepare an Rmarkdown document with the results. At the end of the document write a paragraph to explain your “findings”. As if you were in a debate team, pick on a subjective conclusion, and “cherry-pick” partial data to support your claim. Provide a non-statistical explanation for your group’s fake result, and/or critique your statistical approach and why your group got an apparently significant p-value.

As an example, below on a particular date (02/26/2020), I found a positive relationship between handwashing facilities and new cases in Asia countries.

Some sample code:

```
library(dplyr)
#read the Dataset sheet into "R". The dataset will be called "data".
data <- read.csv("https://covid.ourworldindata.org/data/owid-covid-data.csv",
  na.strings = "",header=T)

res <- NULL
for (i in 1:length(unique(data$date))){
  data1 <- data[which((data$date==unique(data$date)[i])&(data$continent=="Asia")),]
  data1 <- data1 %>% select("iso_code","date","new_cases", "handwashing_facilities")
  if (sum(rowSums(!is.na(data1[,3:4]))==2)>=10){
    res <- rbind(res, c(unique(data$date)[i],
      cor.test(data1[,3],data1[,4])$estimate,
      cor.test(data1[,3],data1[,4])$p.value))
  }}

res[which((as.numeric(res[,2])>0)& (as.numeric(res[,3])<=0.05)),]
```